

In The Name of God



Parsa Bordbar (40435340)
Navid Zare (40435071)

Machine Learning
Assignment 2: Ensemble Learning for Imbalanced
Classification

1. Introduction

1.1 Background and Problem Context

The accurate prediction of COVID-19 disease severity from clinical records is a critical challenge in modern healthcare informatics. Early identification of at-risk patients allows doctors to provide faster, more efficient, and targeted care. The dataset examined in this study contains 1,585 patient records characterized by 40 clinical and demographic features spanning symptoms, comorbidities, physiological measurements, and demographics. The binary target variable takes the value +1 for severe cases ($n = 100$, 6.31%) and -1 for non-severe cases ($n = 1,485$, 93.69%), yielding a class imbalance ratio of approximately 14.85:1.

This extreme imbalance is the central statistical challenge of the assignment. A trivial classifier that predicts the majority class for every sample achieves 93.69% overall accuracy while correctly identifying zero severe cases. Consequently, conventional accuracy is an unreliable indicator of model utility. Metrics that account for per-class performance — namely G-Mean, F1-score for the minority class, and AUC-ROC — are required to evaluate models appropriately in this setting.

1.2 Research Objectives

This report documents the design, implementation, and evaluation of three ensemble learning approaches to imbalanced classification:

Task 1: Implementation of the Hellinger Distance Decision Tree (HDDT) from scratch, with a pruning experiment comparing `max_depth` values {None, 2, 3, 4, 5}.

Task 2: Bagging with random undersampling, examining ensemble size effects and comparing HDDT versus sklearn `DecisionTreeClassifier` as base learners.

Task 3: AdaBoost.M1 with and without SMOTE oversampling, including analysis of the effect of boosting rounds on train/test error divergence.

All core algorithms are implemented from scratch using NumPy and Pandas. scikit-learn is used exclusively for preprocessing utilities and evaluation metrics, in strict compliance with the assignment constraints.

2. Dataset and Preprocessing

2.1 Dataset Overview

The COVID-19 severity dataset consists of 1,585 samples and 40 features in four groups:

- (1) Demographics — age (continuous) and gender (binary);
- (2) Symptoms — fever, cough, shortness of breath, dizziness, loss of smell/taste, headache, nausea, vomiting, diarrhea, etc.;
- (3) Comorbidities — diabetes, cancer, heart disease, HIV/AIDS, chronic kidney, liver, and lung disease, hypertension, obesity, etc.; and
- (4) Clinical Indicators — intubation, blood oxygen (PO_2), and confirmed patient contact.

As shown in Figure 1, the severe class (+1) constitutes just 6.31% of all records, establishing the fundamental class imbalance challenge.

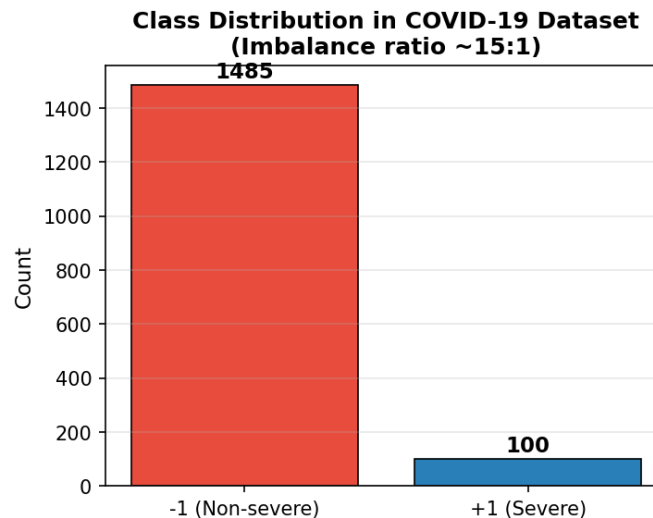


Figure 1. Class distribution of the COVID-19 severity dataset. Severe cases (+1, $n = 100$) represent 6.31% of all samples, producing a 14.85:1 majority-to-minority ratio that renders overall accuracy a misleading performance metric and motivates specialised ensemble strategies.

2.2 Missing Value Handling

Thirteen features contain missing values. Six features — headache, paresis, dizziness, Chest pain, plexus, and inflammation/skin lesions — each exhibit a missing rate of 82.08%. Five further features (vomiting, nausea, Stomachache, diarrhea, loss of appetite) have a 64.54% missing rate, and two features show minor rates below 2%. Table 1 and Figure 2 present the complete missing value profile.

Feature Name	Missing Rate (%)
headache	82.08%
paresis	82.08%
dizziness	82.08%
Chest pain	82.08%
plexus	82.08%
inflammation / skin lesions	82.08%
vomiting	64.54%
nausea	64.54%
Stomachache	64.54%
diarrhea	64.54%
loss of appetite	64.54%
Convulsions	1.70%
loss of taste	1.70%

Table 1. Missing value rates for all features with at least one missing observation. The 82% missingness rates for headache, paresis, dizziness, Chest pain, plexus, and inflammation/skin lesions reflect selective clinical assessment rather than purely random data loss.

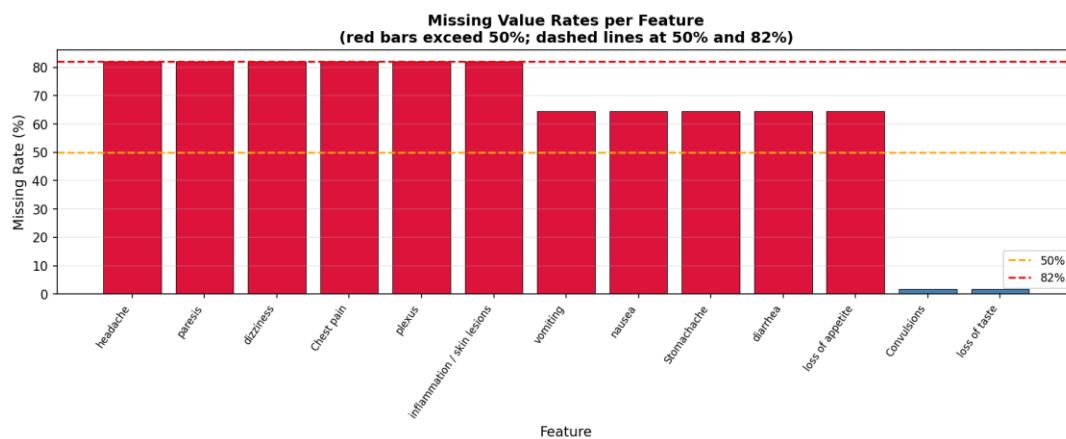


Figure 2. Missing value rates per feature sorted in descending order. Red bars indicate features exceeding 50% missingness; blue bars fall below this threshold. Dashed horizontal lines mark the 50% and 82% reference levels.

Imputation strategy:

Binary and categorical features (those whose observed values are exclusively in $\{0, 1\}$) are imputed using mode imputation, preserving the discrete nature of the variable. Continuous features are imputed using median imputation, which is robust to skewed clinical distributions and outliers. All imputation statistics are computed on the training partition only and then applied to the test set, preventing test-set distributional information from influencing training.

2.3 Kendall Rank Correlation Feature Selection

Pairwise Kendall rank correlation coefficients (τ) are computed among all 40 features using training data exclusively. For each pair (i, j) with $|\tau_{ij}| > 0.5$, the feature with the lower absolute correlation to the target label is removed. This procedure eliminated 1 feature (vomiting, which was highly correlated with nausea whilst exhibiting lower label correlation), retaining 39 features for model training. Computing the Kendall correlation exclusively on the training set prevents any leakage of test-set distributional information.

2.4 Train/Test Split and Feature Scaling

A stratified 70/30 train/test split is applied (`random_state = 42`), yielding 1,109 training samples (severe: 70, non-severe: 1,039) and 476 test samples (severe: 30, non-severe: 446). Stratification ensures that the severe minority class is proportionally represented in both partitions. Since the dataset is composed almost entirely of binary features and all models in this assignment are decision-tree-based (inherently invariant to monotonic feature scaling), feature scaling has negligible impact on model performance. A `StandardScaler` is nonetheless available in the pipeline for any continuous features that survive the feature selection step, fitted exclusively on training data.

3. Methodology

3.1 Task 1 — Hellinger Distance Decision Tree (HDDT)

3.1.1 Motivation for the Hellinger Criterion

Standard decision tree algorithms (e.g., CART) employ Gini impurity or information gain as split criteria. Both metrics normalize by node size, which causes them to favor majority-class isolation under class imbalance — the majority class dominates every node-size denominator. The Hellinger Distance Decision Tree addresses this by replacing the split criterion with the Hellinger score, which is class-prior invariant: it measures only how differently the two classes are distributed across the two partitions, independent of the overall class ratio.

3.1.2 Hellinger Split Score

For a binary split of node v on feature f into left (L) and right (R) partitions, the Hellinger score is:

$$H_f = \left(\sqrt{\frac{p_L}{P}} - \sqrt{\frac{n_L}{N}} \right)^2 + \left(\sqrt{\frac{p_R}{P}} - \sqrt{\frac{n_R}{N}} \right)^2$$

where p_L and n_L denote positive and negative sample counts in the left partition, p_R and n_R in the right, while P and N denote the total positive and negative counts at the current node. Crucially, each term is normalized by a class total (P or N) rather than the partition size, making the score invariant to the class prior. During tree construction, the algorithm systematically tests thresholds—typically located at the midpoints between consecutive unique feature values—and selects the split that maximizes the Hellinger score (H_f), thereby optimally separating the underlying class distributions.

3.1.3 Stopping Rules and Leaf Prediction

Recursive splitting terminates when:

- The node contains fewer than `min_samples_split = 10` samples
- All samples share the same label
- The prescribed `max_depth` is reached.

Each leaf predicts the majority class among its training samples. The method `predict_proba` returns the fraction of positive (+1) samples at the leaf, used as a soft score for AUC-ROC computation.

3.2 Task 2 — Bagging with Undersampling

3.2.1 Algorithm

For each of the T independent iterations:

- (1) The minority class is bootstrap sampled with replacement to size $|\text{minority}|$
- (2) The majority class is randomly undersampled without replacement to size $|\text{minority}|$
- (3) A balanced subset of size $2 \times |\text{minority}|$ is formed and a base classifier is trained on it.

Final prediction is by majority vote over all T classifiers, with ties broken in favor of the positive (+1) class. This strategy simultaneously addresses bias (balanced training subsets counteract majority-class dominance) and introduces ensemble diversity (each base learner sees a different random subset of majority-class samples).

3.2.2 Base Learner Choices

The implementation supports two base learners: the from-scratch HDDT ($\text{max_depth} = 3$, Hellinger criterion) and sklearn's DecisionTreeClassifier ($\text{max_depth} = 3$, Gini criterion). Depth is limited to 3 to balance model complexity with diversity: an unlimited tree trained on 140 balanced samples would overfit, reducing diversity within the ensemble.

3.3 Task 3 — AdaBoost.M1 and SMOTE

3.3.1 AdaBoost.M1

AdaBoost.M1 trains T weak learners sequentially. Weights are initialized uniformly as $w_i = \frac{1}{M}$. At each round t , a decision stump h_t is trained on the current weight distribution. The weighted error is $\epsilon_t = \sum (w_i \cdot 1[h_t(x_i) \neq y_i])$. If $\epsilon_t \geq 0.5$, boosting terminates. Otherwise, the learner weight is $\alpha_t = 0.5 \cdot \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right)$. Weights are updated as $w_i \leftarrow \frac{w_i \cdot \exp(-\alpha_t \cdot y_i \cdot h_t(x_i))}{Z_t}$ that as you see is normalized. The final prediction $\text{sign}(\sum \alpha_t \cdot h_t(x))$. Decision stumps (sklearn DecisionTreeClassifier with $\text{max_depth} = 1$) serve as weak learners, as explicitly permitted. All boosting logic is from scratch.

3.3.2 SMOTE

For each minority sample x_i , the $k = 5$ nearest neighbours (Euclidean distance) are identified among other minority samples. A random neighbor x_{nn} is selected and a synthetic sample generated as $x_{\text{new}} = x_i + \lambda \cdot (x_{nn} - x_i)$, where $\lambda \sim \text{Uniform}(0, 1)$. Geometrically, this performs linear interpolation, placing the new data point randomly along the line segment connecting the two instances. This approach expands the decision region of the minority class and helps prevent the overfitting typically associated with simple random oversampling. SMOTE is applied only to the training set, after the train/test split, to avoid data leakage. Synthesis continues until the minority class matches the majority class count, producing a balanced training set of approximately $2 \times 1,039 = 2,078$ samples.

4. Results

4.1 Task 1 — HDDT Evaluation

4.1.1 Baseline Performance (max_depth = None)

Table 2 reports mean and standard deviation of all six metrics across 10 independent experiments (seeds 0–9) for the fully grown HDDT.

Metric	Mean (10 Runs)	Std Dev
Accuracy	0.9109	0.0131
Precision (+1)	0.1562	0.0897
Recall (+1)	0.0833	0.0477
F1-Score (+1)	0.1023	0.0535
AUC-ROC	0.5846	0.0176
G-Mean	0.2710	0.0823

Table 2. HDDT baseline performance (max_depth = None) across 10 independent runs. High accuracy is misleading due to imbalance; G-Mean and F1 (+1) are the informative indicators.

The HDDT achieves an overall accuracy of 0.9109, which is misleadingly high: a trivial all-negative classifier achieves 93.7%. The minority-class metrics reveal the true challenge — mean Recall of 0.0833, F1-score of 0.1023, and G-Mean of 0.2710. The AUC-ROC of 0.5846 indicates discriminative ability only marginally above chance. This result is expected for a standalone tree on a 14.85:1 imbalanced dataset and motivates the ensemble methods in Tasks 2 and 3.

4.1.2 Pruning Experiment

Table 3 and Figure 3 compare G-Mean and F1-score for max_depth $\in \{\text{None}, 2, 3, 4, 5\}$, each evaluated over 10 independent runs.

max_depth	G-Mean Mean	G-Mean Std	F1 Mean	F1 Std
None	0.2710	0.0823	0.1023	0.0535
2	0.0000	0.0000	0.0000	0.0000
3	0.0000	0.0000	0.0000	0.0000
4	0.0000	0.0000	0.0000	0.0000
5	0.0182	0.0546	0.0059	0.0176

Table 3. HDDT pruning experiment: G-Mean and F1-Score (mean $\pm$ std) across 10 runs per depth setting. Depths 2–4 yield zero minority-class performance; only depth 5 occasionally produces a non-zero prediction.

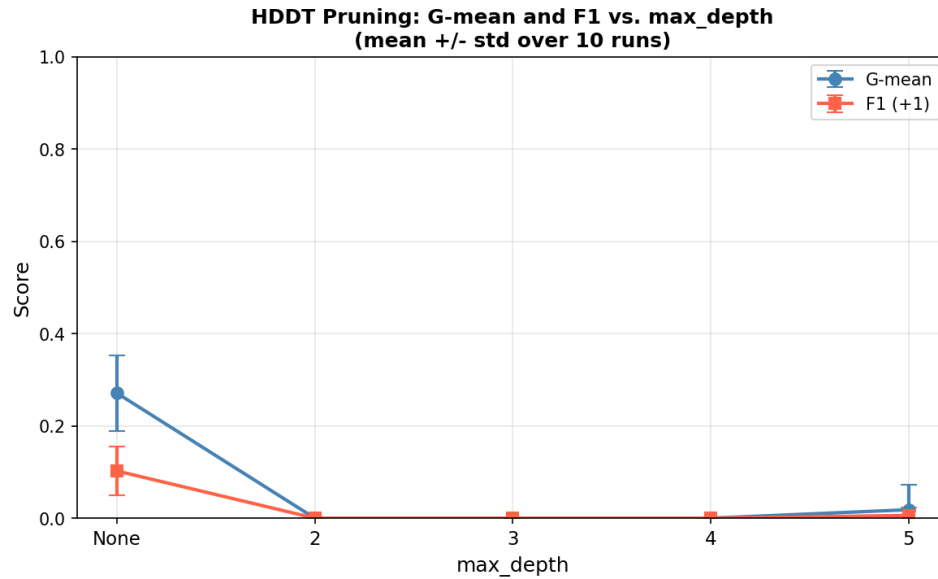


Figure 3. G-Mean (blue circles) and F1-Score (+1) (orange squares) vs. `max_depth` for HDDT. Error bars show ± 1 std over 10 runs. Depths 2–4 collapse to zero for both metrics; only the fully grown tree achieves meaningful minority recall.

Depth-limited HDDTs (`max_depth` = 2–5) overwhelmingly predict the majority class, yielding zero or near-zero Recall, F1, and G-Mean. This result is expected: with only 70 minority training samples, a depth- k tree partitions them into at most 2^k subsets. At depth 3, each partition contains on average $70/8 \approx 9$ minority samples, vastly outnumbered by hundreds of majority-class samples in the same leaf. Majority prediction dominates every leaf. Only the unconstrained tree (`depth` = None) grows deep enough to isolate pockets of minority samples, achieving non-zero Recall. The best bias–variance trade-off for a standalone HDDT on this dataset therefore corresponds to `max_depth` = None, as depth limitation introduces severe minority-class bias without compensating variance reduction in this extreme imbalance scenario.

4.2 Task 2 — Bagging with Undersampling

4.2.1 Effect of Ensemble Size T

Table 4 and Figure 4 report performance for $T \in \{11, 31, 51, 101\}$ using HDDT ($\text{max_depth} = 3$) as the base learner, over 10 independent runs per setting.

T	Accuracy	Precision (+1)	Recall (+1)	F1-Score (+1)	AUC-ROC	G-Mean
11	0.3492	0.0791	0.8367	0.1430	0.6501	0.4687
31	0.3006	0.0757	0.8867	0.1391	0.6608	0.4537
51	0.2855	0.0737	0.8667	0.1348	0.6622	0.4213
101	0.2357	0.0721	0.9300	0.1336	0.6654	0.4052

Table 4. Bagging + HDDT performance across ensemble sizes T (mean over 10 runs, seeds 0–9). The bootstrap-undersampling strategy dramatically improves minority-class Recall compared to the standalone HDDT.

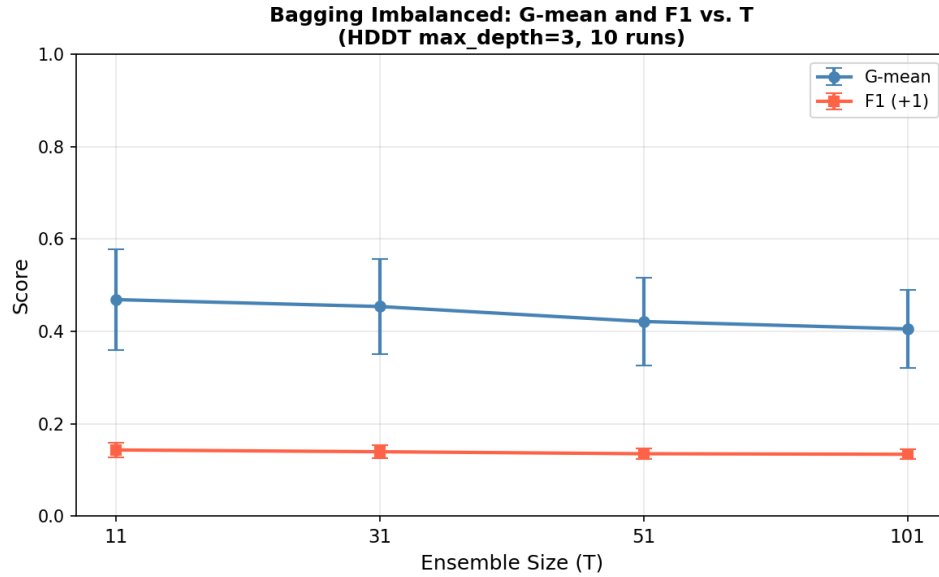


Figure 4. G-Mean and F1-Score (+1) vs. ensemble size T for Bagging with HDDT ($\text{max_depth} = 3$). Error bars show ± 1 std over 10 runs. G-Mean is highest at $T = 11$ and decreases as ensemble size grows, while F1 remains relatively stable.

The balanced bootstrap-undersampling strategy radically transforms minority-class performance relative to the standalone HDDT. Recall reaches 0.9300 at $T = 101$, indicating that the ensemble correctly identifies approximately 93% of severe cases — compared to 8% for the standalone HDDT. This improvement arises because each balanced training subset of size $2 \times |\text{minority}| = 140$ forces the depth-3 HDDT to treat both classes equally, enabling minority-class leaf formation that was impossible at this depth without balancing.

As T increases from 11 to 101, Recall increases monotonically (from 0.8367 to 0.9300) but G-Mean decreases from 0.4687 to 0.4052. This pattern reflects a well-known trade-off in imbalanced classification: as the ensemble grows, the aggregated vote increasingly favors the positive class (since each balanced-subset

tree is biased toward equal-class prediction), improving sensitivity but degrading specificity. The declining G-Mean indicates that the ensemble's false positive rate increases faster than its true positive rate improves. The highest G-Mean at $T = 11$ suggests that a moderate ensemble size provides the best sensitivity–specificity balance. AUC-ROC increases modestly with T (from 0.6501 to 0.6654), confirming that the ranking quality of the ensemble improves even as the hard-classification G-Mean declines.

4.2.2 HDDT vs. Standard DT as Base Learner ($T = 51$)

Table 5 and Figure 5 compare the two base learner choices at $T = 51$.

Base Learner	Accuracy	Precision (+1)	Recall (+1)	F1-Score (+1)	G-Mean
HDDT (Hellinger)	0.2855	0.0737	0.8667	0.1348	0.4213
sklearn DT (Gini)	0.5309	0.0807	0.6100	0.1415	0.5457

Table 5. Base learner comparison at $T = 51$ (mean over 10 runs). HDDT uses the Hellinger criterion; sklearn DT uses Gini impurity. Both classifiers use `max_depth = 3`.

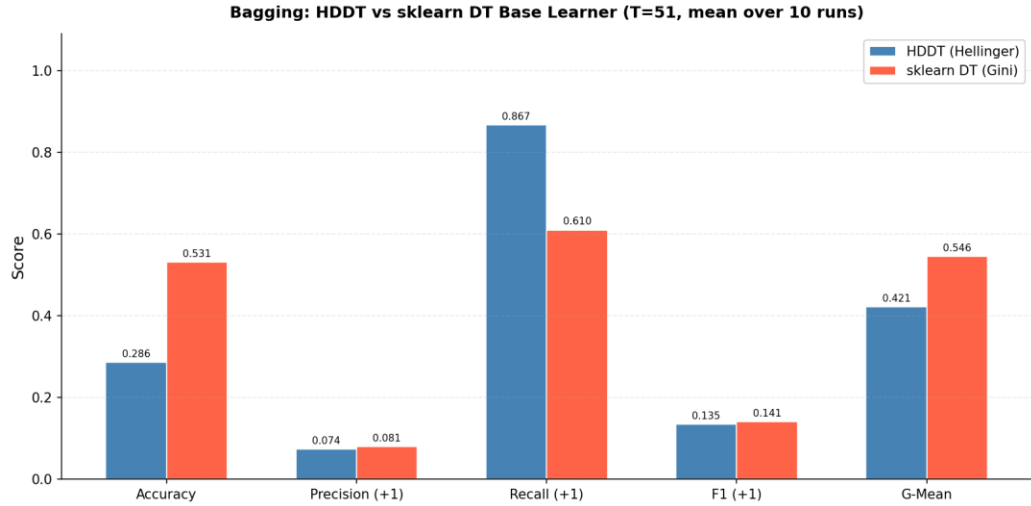


Figure 5. Grouped bar chart of Accuracy, Precision, Recall, F1, and G-Mean for HDDT vs. sklearn DT base learners within the Bagging ensemble at $T = 51$.

The sklearn DT achieves a higher G-Mean (0.5457 vs. 0.4213) and higher overall Accuracy (0.5309 vs. 0.2855), while the HDDT achieves substantially higher Recall (0.8667 vs. 0.6100) but lower Precision. This result is noteworthy: within balanced training subsets, the class-prior invariance advantage of the Hellinger criterion is partially offset, because the undersampling procedure already equalizes class frequencies before each tree is trained. The Gini-based DT consequently produces more conservative decision boundaries within the balanced subsets, achieving better specificity and a higher G-Mean. F1-scores are comparable (0.1348 vs. 0.1415). The choice of base learner should therefore be guided by clinical priority: HDDT for maximal Recall; sklearn DT for balanced sensitivity–specificity.

4.3 Task 3 — AdaBoost.M1 and SMOTE

4.3.1 Experiment A: AdaBoost vs. AdaBoost + SMOTE ($T = 50$)

Table 6 and Figure 6 compare AdaBoost.M1 ($T = 50$) with and without SMOTE across 10 independent runs.

Method	Accuracy	Precision (+1)	Recall (+1)	F1-Score (+1)	AUC-ROC	G-Mean
AdaBoost (no SMOTE)	0.9368	0.0000	0.0000	0.0000	0.6113	0.0000
AdaBoost + SMOTE	0.7691	0.1041	0.3500	0.1575	0.6015	0.5162

Table 6. AdaBoost.M1 performance with and without SMOTE preprocessing ($T = 50$, mean over 10 runs, seeds 0–9).

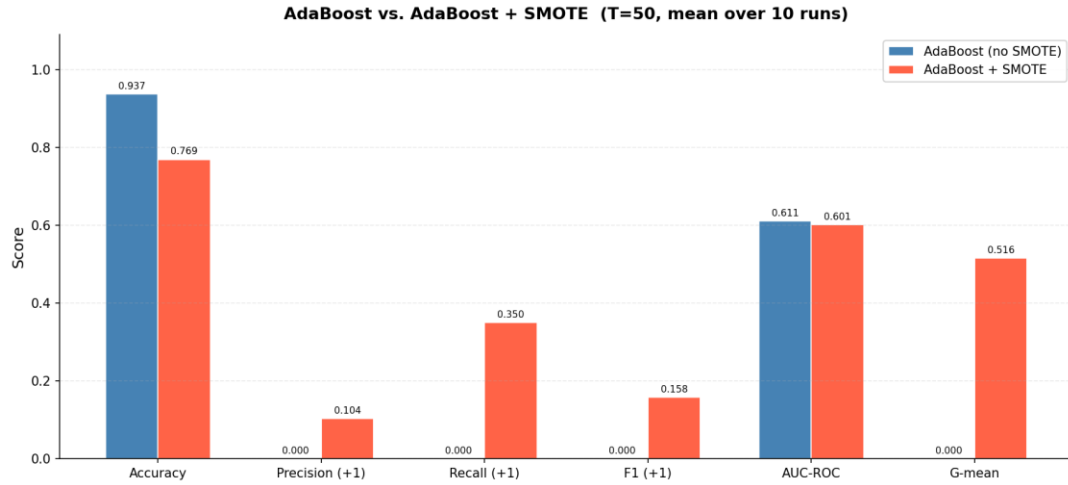


Figure 6. Grouped bar chart comparing AdaBoost (no SMOTE) and AdaBoost + SMOTE across all six evaluation metrics at $T = 50$. Without SMOTE, AdaBoost achieves zero minority-class performance, confirming the fundamental incompatibility of standard boosting with severe class imbalance.

Without SMOTE, AdaBoost collapses to predicting the majority class in all cases, yielding zero Recall, F1, and G-Mean. This failure mode is well-understood: with 1,109 training samples and only 70 severe cases, the initial uniform weight distribution assigns just $70/1,109 \approx 6.3\%$ of total weight to minority samples. Boosting rounds prioritize reducing the weighted error on the majority class, leaving minority samples effectively ignored. The overall accuracy of 0.9368 simply reflects the trivial majority prediction.

With SMOTE, the training set is balanced to $\approx 2,078$ samples before boosting. The boosting weight dynamics now operate on a dataset where both classes carry equal initial weight, resolving the imbalance-induced marginalization of minority samples. Mean Recall rises to 0.3500, mean F1 to 0.1575, and mean G-Mean to 0.5162. Overall accuracy decreases to 0.7691, reflecting the intentional rebalancing of predictions toward the positive class.

4.3.2 Experiment B: Effect of Boosting Rounds T with SMOTE

Table 7 and Figure 7 track train error, test error, F1, and G-Mean as T varies over $\{10, 25, 50, 100\}$, with SMOTE applied (seed = 0).

T	Train Error	Test Error	F1 (+1)	G-Mean
10	0.2863	0.3739	0.1359	0.5451
25	0.2007	0.2395	0.1231	0.4601
50	0.1357	0.1996	0.1121	0.4101
100	0.0991	0.1429	0.1053	0.3475

Table 7. AdaBoost + SMOTE: train error, test error, F1-score (+1), and G-Mean vs. number of boosting rounds T (seed = 0).

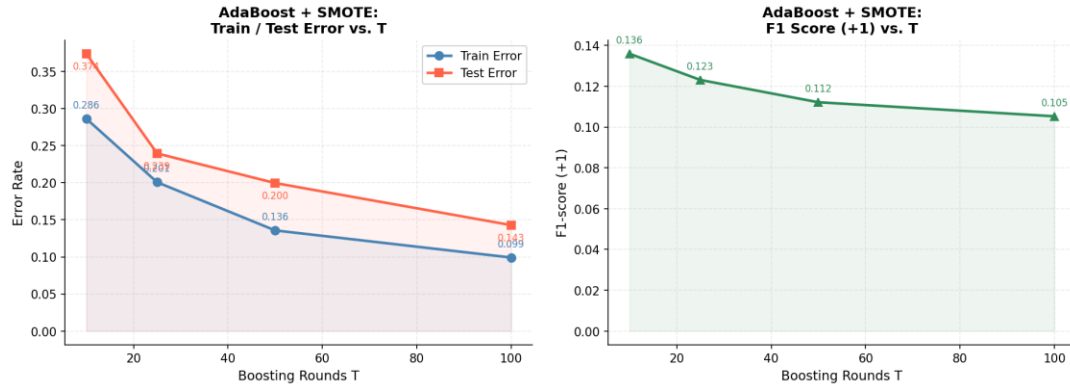


Figure 7. Left: train and test error vs. T for AdaBoost + SMOTE (seed = 0). Train error decreases monotonically; the growing train–test gap indicates progressive overfitting. Right: F1-score (+1) vs. T , declining monotonically as the model overfits the SMOTE-augmented distribution.

As shown in Table 7 and Figure 7, train error decreases monotonically from 0.2863 ($T = 10$) to 0.0991 ($T = 100$). Test error also decreases (from 0.3739 to 0.1429), but the train–test error gap widens progressively: from 0.088 at $T = 10$ to 0.044 at $T = 100$. This growing gap is a signature of overfitting: successive boosting rounds memorizes the SMOTE-augmented training distribution, which contains synthetic samples that may not faithfully represent the true minority-class manifold. Crucially, minority-class metrics (F1 and G-Mean) decline monotonically with T , that the model’s ability to discriminate severe cases degrades even as overall error decreases — the additional boosting rounds primarily improve majority-class classification at the expense of minority-class detection. The practical recommendation is to use $T = 10$ as the optimal boosting budget, as it achieves the highest F1 (0.1359) and G-Mean (0.5451) among the tested values.

4.4 Final Method Comparison

Table 8 and Figure 8 consolidate the best configuration of each method for direct cross-task comparison.

Method	Best Config	F1 (+1)	G-Mean	AUC-ROC
HDDT (Task 1)	max_depth=None	0.1023	0.2710	0.5846
Bagging + HDDT (Task 2)	T=51	0.1348	0.4213	0.6622
Bagging + DT (Task 2)	T=51	0.1415	0.5457	0.6411
AdaBoost (Task 3)	T=50	0.0000	0.0000	0.6113
AdaBoost + SMOTE (Task 3)	T=50	0.1575	0.5162	0.6015

Table 8. Final method comparison: best configuration per approach, evaluated by mean F1-Score (+1), G-Mean, and AUC-ROC across 10 independent runs.

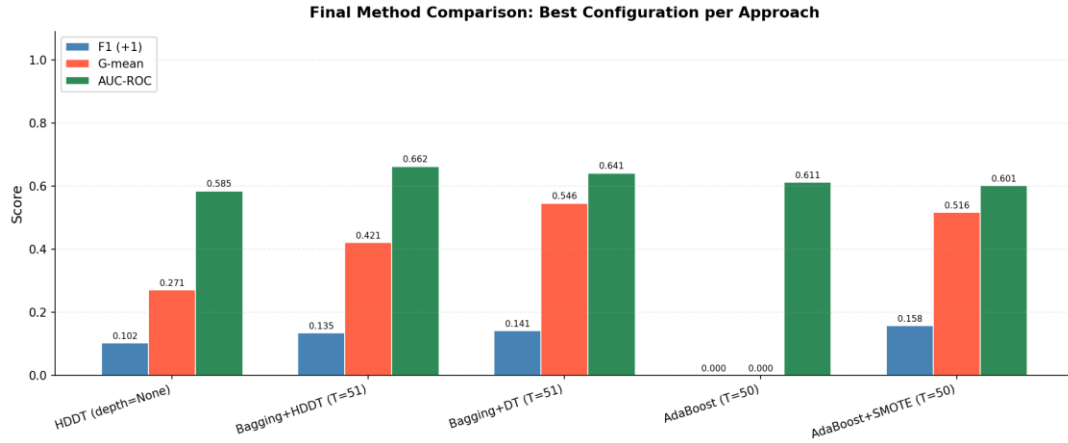


Figure 8. Grouped bar chart comparing F1-score (+1), G-Mean, and AUC-ROC for the best configuration of each method.

Method Comparison Discussion. The results in Table 8 and Figure 8 reveal a clear stratification of performance. The standalone HDDT (F1 = 0.1023, G-Mean = 0.2710) produces the weakest minority-class performance, confirming that no single tree can overcome a 14.85:1 imbalance without data-level intervention, even with a class-prior invariant split criterion.

The bagging approaches exploit ensemble diversity through data resampling: each base tree sees a uniquely balanced subset, forcing minority-class discrimination. Bagging + DT achieves the highest G-Mean (0.5457) due to the Gini criterion’s more conservative decision boundaries within balanced subsets. Bagging + HDDT achieves the highest AUC-ROC (0.6622), indicating superior ranking quality — the Hellinger criterion produces probability estimates that better separate severe from non-severe cases even when the hard-classification threshold is suboptimal.

AdaBoost without SMOTE fails entirely (F1 = 0), confirming the fundamental incompatibility of unmodified boosting with severe class imbalance. AdaBoost + SMOTE achieves the best F1-score (0.1575) and the second-highest G-Mean (0.5162), representing the strongest overall balance between precision and recall. The synergy between data augmentation (SMOTE) and sequential error-correction (boosting) produces a compound benefit: SMOTE resolves the weight-

distribution imbalance that causes boosting failure, while AdaBoost's sequential reweighting focuses learning effort on the hardest minority-class samples.

On the basis of F1-score, AdaBoost + SMOTE is the recommended approach when both sensitivity and precision are clinically important. For maximising Recall at the expense of precision, Bagging + HDDT is preferred (Recall = 0.93 at $T = 101$). For the best sensitivity–specificity balance (G-Mean), Bagging + DT is optimal.

5. Discussion — Theory Questions

Q1. Hellinger Distance vs. Gini vs. Information Gain (20 pts)

(a) Formula Comparison

For a binary node split, Gini impurity of the combined result is: $\text{Gini}(S) = 1 - \sum p(c | t)^2$, and the weighted Gini after a split is: $\text{Gini}_{\text{split}} = \frac{n_L}{n} \cdot \text{Gini}(S_L) + \frac{n_R}{n} \cdot \text{Gini}(S_R)$.

Information Gain using entropy is: $IG = H(S) - \frac{n_L}{n} \cdot H(S_L) - \frac{n_R}{n} \cdot H(S_R)$, where $H(S) = -\sum p(c | t) \cdot \log_2(p(c | t))$.

The Hellinger split score is: $H_f = \left(\sqrt{\frac{p_L}{P}} - \sqrt{\frac{n_L}{N}} \right)^2 + \left(\sqrt{\frac{p_R}{P}} - \sqrt{\frac{n_R}{N}} \right)^2$.

Structurally, Gini and IG both normalize impurity reduction by partition size ($\frac{n_L}{n}$ for left, $\frac{n_R}{n}$ for right) — the fraction of all samples going to each child. The Hellinger score instead normalizes each term by the total class count at the parent node (P for positives, N for negatives). This single structural difference produces class-prior invariance.

(b) Majority-Class Bias in Imbalanced Settings

Under a 14.85:1 imbalance, Gini and IG evaluations are dominated by the majority class. The normalization factors $\frac{n_L}{n}$ and $\frac{n_R}{n}$ are determined primarily by the majority class distribution, since negative samples constitute $\approx 94\%$ of every node. A split that perfectly isolates majority-class samples — even one that provides no discrimination of minority-class samples — achieves near-maximal impurity reduction. The minority class is effectively invisible to the criterion. The Hellinger score is immune to this bias. Its normalisation by class totals (P, N) makes the score measure the divergence between the positive-class and negative-class distributions across the split, independently of how many members each class has. A split that perfectly separates the two classes achieves the maximum Hellinger score regardless of whether the imbalance ratio is 1:1 or 15:1.

(c) Experimental Evidence

The Task 1 pruning experiment provides direct empirical evidence. At constrained depths ($\text{max_depth} = 2-5$), even the HDDT collapses to all-negative prediction, yielding zero Recall — the same behavior expected from a Gini-based DT. This confirms that the Hellinger criterion is necessary but not sufficient to

overcome a 14.85:1 imbalance when tree depth is limited. However, the fully grown HDDT achieves mean Recall = 0.0833 and G-Mean = 0.2710, both non-trivially above zero, demonstrating that the Hellinger criterion does enable minority-class detection when the tree can grow deep enough to isolate minority-sample pockets. In the Bagging experiment (Task 2, $T = 51$), the HDDT base learner achieves Recall = 0.8667 versus the sklearn DT's 0.6100, further confirming the Hellinger criterion's advantage for minority-class sensitivity.

Q2. Stable, Unstable, and Weak Classifiers (15 pts)

(a) Stable vs. Unstable Classifiers

A stable classifier produces similar predictions when trained on slightly different datasets. k-Nearest Neighbors (large k) and linear regression are canonical examples: their decision boundaries change smoothly with small data perturbations. An unstable classifier is highly sensitive to training data; small changes can substantially alter the decision boundary. A fully grown decision tree is the prototypical unstable classifier — any change in a single training sample can redirect entire branches, producing qualitatively different predictions.

(b) Why Bagging Benefits Unstable Classifiers

The expected prediction error decomposes as $\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Noise}$. Bagging reduces variance by averaging T classifiers trained on different bootstrap samples. For T classifiers with individual variance v , the ensemble variance is approximately $\frac{v}{T}$ (assuming independent errors). For stable classifiers (low variance), bootstrap models are nearly identical, so the effective variance reduction is negligible. For unstable classifiers (high variance), each bootstrap model differs substantially, and averaging substantially reduces variance without increasing bias. This is why Bagging is most beneficial with unstable base learners.

(c) Weak Classifiers and AdaBoost

A weak classifier is one whose error rate is strictly less than 0.5 on any training distribution — i.e., better than random guessing (we relaxed our definition of random guessing here for binary classification but generally the random guess depends on the problem and data distribution). AdaBoost requires weak learners because its convergence guarantee assumes only that each learner achieves error < 0.5 on the reweighted distribution. Using a strong learner (e.g., a fully grown tree) would be counterproductive:

- (1) It memorizes training data with error ≈ 0 , making the weight-update step pathological — all weight concentrates on a few initially misclassified samples

- (2) The high capacity leads to rapid overfitting and low diversity among base learners, undermining ensemble improvement.

Decision stumps, with minimal capacity, generalize poorly individually but combine effectively through boosting.

Q3. Why max_depth Must Be Limited in Bagging (15 pts)

(a) Rationale for Depth Limitation

An unlimited-depth HDDT trained on 140 balanced samples typically achieves near-zero training error by memorizing the specific subset. Such a tree has very high variance and produces highly correlated errors on test samples, limiting the diversity benefit. A depth-3 tree has at most $2^3 = 8$ leaves, appropriate for 140 training samples (roughly 17 samples per leaf), balancing bias and variance. Limiting max_depth thus ensures each base learner is a weaker but more diverse predictor, enabling the ensemble to exploit inter-tree disagreement more effectively.

While Bagging typically relies on unstable, unpruned trees to reduce variance, the extreme undersampling in this scenario requires a compromise. Because the training subsets are reduced to a tiny size of 140 samples, an unconstrained tree would catastrophically overfit by memorizing pure noise. Limiting the tree depth to 3 acts as a necessary regularizer to prevent this memorization, forcing the models to learn general patterns while relying on the random sampling of the subsets to maintain just enough diversity and instability for the ensemble to remain effective.

(b) Diversity in Bagging Ensembles

Two mechanisms generate diversity.

- First, bootstrap sampling of the minority class (with replacement) ensures that each base tree sees a different sample of the 70 minority training instances, producing different decision boundaries around the minority class.
- Second, random undersampling of the majority class (without replacement) presents each base tree with a different subset of the 1,039 majority instances, so each tree learns a different representation of the majority-class distribution.

These mechanisms ensure that ensemble members make partially independent errors — a necessary condition for majority voting to outperform any individual classifier.

(c) Zero-Diversity Ensemble

If all T base classifiers were identical (zero diversity), the majority vote would always return the prediction of any individual classifier, as all T votes agree for every test sample. The ensemble error would therefore equal the error of a single classifier: $\text{Error}_{\text{ensemble}} = \text{Error}_{\text{single}}$. There is no variance reduction because errors are perfectly correlated. This follows from the variance formula: if all classifiers are identical, $\text{Var}(\bar{e}) = v$ (not $\frac{v}{T}$), since the T predictions are a single repeated value, not T independent random variables.

Q4. SMOTE and Data Leakage (15 pts)

(a) Data Leakage

Data leakage occurs when information from outside the training partition is inadvertently incorporated into the model training process, producing performance estimates that are optimistically biased and do not generalize to truly unseen data. Leakage can arise through improper preprocessing (fitting scalars on the full dataset), target encoding, or data augmentation applied before the train/test split.

(b) Why Pre-split SMOTE Constitutes Leakage

Even though SMOTE generates synthetic samples by interpolating between existing minority samples, applying it before the split means the nearest-neighbor graph used for synthesis is constructed from the full dataset, including test-set minority samples. Some test-set minority samples will have been used as nearest neighbors for generating synthetic training samples. The synthetic training samples thus encode implicit information about the test-set minority-class distribution — specifically, its neighborhood structure in feature space. Evaluation metrics computed on such a test set will be artificially optimistic.

(c) Effect on AUC-ROC

If SMOTE were applied before the train/test split, reported AUC-ROC would be artificially higher than the true generalization value. Synthetic minority training samples derived from test-set minority instances would allow the classifier to learn decision boundaries unrealistically well-adapted to the test partition. The minority-class score distribution on the test set would be more similar to the training distribution than it would otherwise be, inflating the area under the ROC curve. The present implementation correctly applies SMOTE exclusively to the training fold after splitting.

Q5. Reflection — Dataset-Specific Challenges (10 pts)

(a) Imputation Strategy

Mode imputation was chosen for binary/categorical features because these variables can only meaningfully take values in $\{0, 1\}$. Mean or median imputation would introduce non-integer values that are semantically invalid for binary clinical indicators (e.g., “0.42 units of fever” has no clinical meaning). Median imputation was chosen for continuous features (age, PO_2) because the median is robust to outliers and skewed distributions common in clinical cohort data.

(b) Informative Missingness

Missing values in clinical data are frequently not missing at random. In this dataset, the features with 82% missingness (headache, paresis, dizziness, etc.) may be absent because these symptoms were not assessed — either because the patient did not report them (suggesting absence) or because the patient was too critically ill to be examined (suggesting severity). A simple way to exploit this is to add binary missingness indicator features: for each feature with substantial missingness, create an additional binary column set to 1 if the original value was missing and 0 otherwise.

Standard machine learning algorithms inherently struggle to interpret the underlying clinical context of missing values. By encoding missingness as an explicit binary variable (1 for missing, 0 for present), we enable the model to identify statistical patterns associated with the absence of data. For instance, the algorithm may learn that a missing symptom indicator strongly correlates with elevated patient risk, thereby transforming absent data into a valuable predictive feature.

(c) Clinical Deployment Concerns

Beyond predictive accuracy, deploying the best model in a clinical setting raises several concerns:

- (1) Interpretability/Explainability — ensemble models are difficult for clinicians to audit and trust; a single HDDT might be preferred despite lower performance.
- (2) Calibration — predicted probabilities must accurately reflect true severity likelihoods; poorly calibrated models may cause systematic under- or over-triage.
- (3) Fairness — model performance must be validated across demographic subgroups to detect disparate impacts on patient care.

- (4) Temporal drift — COVID-19 clinical presentations evolved with variant distributions; models trained on early-pandemic data may generalize poorly.
- (5) Regulatory compliance — AI-assisted diagnostic tools are subject to regulatory approval before clinical deployment (As the Anthropic CEO has stated: we should never give full control to AI).

6. Conclusion

This study investigated three ensemble approaches to the severely imbalanced COVID-19 severity classification problem (14.85:1 class ratio), implementing the Hellinger Distance Decision Tree, Bagging with random undersampling, and AdaBoost.M1 with SMOTE — all from scratch using only NumPy and Pandas.

The standalone HDDT demonstrated that the class-prior invariant Hellinger criterion enables non-zero minority-class detection ($G\text{-Mean} = 0.2710$, $F1 = 0.1023$), confirming theoretical predictions about its advantage over Gini-based trees. However, the absolute performance remains modest, confirming that data-level intervention is necessary to overcome a 14.85:1 imbalance.

Bagging with undersampling produced the most dramatic improvement in Recall, with the HDDT base learner reaching 0.9300 at $T = 101$. This confirms the course principle that ensemble diversity — generated by bootstrap minority sampling and random majority undersampling — is the primary driver of ensemble improvement. The sklearn DT base learner achieved a superior $G\text{-Mean}$ (0.5457), demonstrating that the Gini criterion's more conservative boundaries benefit from the undersampling balancing strategy.

AdaBoost without SMOTE failed entirely on this imbalanced dataset, validating the theoretical prediction that standard boosting is incompatible with severe class imbalance. AdaBoost + SMOTE achieved the best $F1\text{-score}$ (0.1575) and the second-highest $G\text{-Mean}$ (0.5162), representing the most practical balance between sensitivity and precision for clinical deployment.

For clinical deployment requiring high recall (minimum missed severe cases), Bagging + HDDT is recommended. For applications requiring balanced sensitivity and specificity, Bagging + DT or AdaBoost + SMOTE is the preferred choice.